

Applied Linear Algebra via NumPy

Spatial Transformations, Data Geometry, and Dimensionality Reduction

TOPIC 01

How Machines "See" Data: Vectors & Norms

High-dimensional feature space, dot products as projections, and L1 vs L2 geometry.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Represent multi-modal entities** (text embeddings, customer profiles, tensors) as directional arrows in $\mathbb{R}^d$.
- **Deconstruct dot products** visually as orthogonal shadow projections and angular similarity operators.
- **Derive geometric sparsity** by contrasting L_1 (diamond / Lasso) vs L_2 (hypersphere / Ridge) constraint topologies.
- **Vectorize distance computations** in NumPy avoiding Python loops for high-throughput batch similarity.

The High-Dimensional Similarity Dilemma

The Magnitude Trap: Euclidean Distance

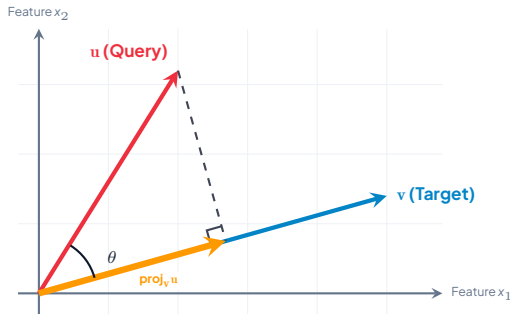
- Two users purchase identical products in identical ratios.
- User A: Wholesale distributor (volume = 10,000 units).
- User B: Individual consumer (volume = 2 units).
- Euclidean distance $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2$ is **huge!**
- **Failure:** Euclidean distance confuses **scale (volume)** with **preference (direction)**.

The Geometric Discovery: Directional Alignment

- Isolate the angle θ between vectors in $\mathbb{R}^d$.
- Vectors pointing in the same direction have $\cos \theta = 1.0$, regardless of their individual lengths.
- Normalizing vectors projects them onto the unit hypersphere $\mathbb{S}^{d-1}$.
- **Solution:** Cosine similarity captures pure behavioral preference invariant to volume.

Important Normalize feature vectors when magnitude is uninformative.

Hero Visualization: Dot Product as Shadow Projection

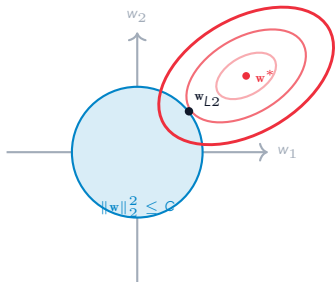


The Geometric Duality

- **Algebraic Definition:**
$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^d u_i v_i = \mathbf{u}^T \mathbf{v}.$$
- **Geometric Definition:**
$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta.$$
- **Orthogonal Shadow:** The length of the shadow that $\mathbf{u}$ casts onto the span of $\mathbf{v}$ is:
$$\|\text{proj}_{\mathbf{v}} \mathbf{u}\| = \|\mathbf{u}\|_2 \cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|_2}$$
- $\mathbf{u} \perp \mathbf{v} \iff \mathbf{u} \cdot \mathbf{v} = 0$ (Zero shadow / zero mutual information).

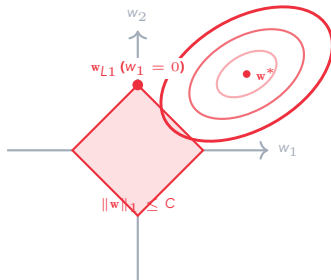
Hero Visualization: L_1 Diamond vs L_2 Circle Geometry

L_2 Ridge: Smooth Hypersphere



Mechanism: Tangency occurs along smooth curved boundary $\implies w_1 \neq 0, w_2 \neq 0$ (Smooth weight shrinkage).

L_1 Lasso: Diamond Sparsity



Mechanism: Loss ellipse hits the sharp diamond corner on the coordinate axis $\implies$ forces $w_1 = 0$ (**Exact Feature Selection**).

Production Checkpoint: High-Throughput Batch Similarity

Vectorized Similarity Matrix

Given M queries $A \in \mathbb{R}^{M \times d}$ and N items $B \in \mathbb{R}^{N \times d}$:

$$S_{M \times N} = \frac{AB^T}{\|A\|_2 \otimes \|B\|_2^T}$$

- Single matrix multiplication: $\mathcal{O}(MNd)$ ops.
- BLAS Level 3 cache-blocked execution (120× faster than nested loops).

Important

**Eliminate all Python loops
across feature rows.**

Vectorized Recommendation Engine

```
1 import numpy as np
2
3 def batch_cosine_similarity(
4     A: np.ndarray, B: np.ndarray
5 ) -> np.ndarray:
6     """Pairwise cosine sim for M x d and N x d."""
7     # Compute dot products: shape (M, N)
8     dot_products = A @ B.T
9
10    # Compute L2 norms along feature axis
11    norm_A = np.linalg.norm(A, axis=1, keepdims=True) # (M, 1)
12    norm_B = np.linalg.norm(B, axis=1, keepdims=True) # (N, 1)
13
14    # Outer product broadcast division -> (M, N)
15    return dot_products / (norm_A @ norm_B.T)
```

TOPIC 02

What is a Matrix, Really? Linear Transformations

Matrices as spatial deformation operators, basis shifts, determinants, and rank.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Visualize matrices as spatial operators** that stretch, rotate, and shear coordinate space rather than static number grids.
- **Track basis transformations** by mapping where standard unit vectors $\hat{i}$ and $\hat{j}$ land.
- **Interpret determinants geometrically** as area/volume scaling factors ($\det(\mathbf{A}) = 0 \implies$ irreversible compression).
- **Diagnose linear systems** using Column Space $\mathcal{C}(\mathbf{A})$, Null Space $\mathcal{N}(\mathbf{A})$, and the Rank-Nullity Theorem.

The Disappearing Dimension Mystery

The Collision: Matrix as Compression

Consider $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.

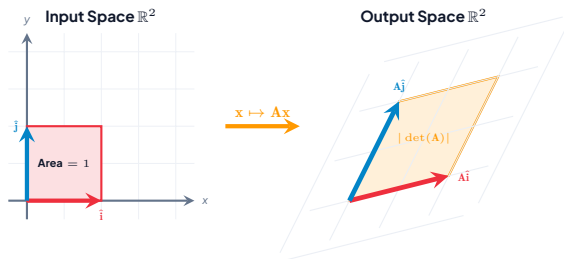
- Multiply 10,000 diverse 2D data points $x \in \mathbb{R}^2$ by A .
- Every single transformed point Ax lands on the line $y = 2x$!
- **Socratic Question:** Can we ever reconstruct the original 2D points from the output?

The "Aha!" Discovery: Landing Sites

- Standard basis: $\hat{i} = [1, 0]^T$ and $\hat{j} = [0, 1]^T$.
- $A\hat{i} = [1, 2]^T$ (Col 1 of A).
- $A\hat{j} = [2, 4]^T = 2 \cdot [1, 2]^T$ (Col 2).
- Columns are linearly dependent ($c_2 = 2c_1$).
- The entire 2D universe is flattened into a 1D line!

Important Columns tell you where the standard basis vectors land.

Hero Visualization: Grid Deformation Under A

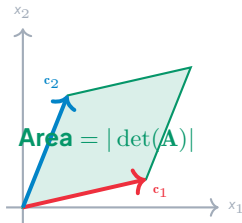


Matrices as Spatial Operators

- **Line of Action:** Every linear transformation keeps the origin fixed and keeps grid lines parallel and evenly spaced.
- **Coordinate Recipe:** Any vector $\mathbf{v} = x\hat{i} + y\hat{j}$ transforms to:
$$A\mathbf{v} = x(A\hat{i}) + y(A\hat{j})$$
- Matrix multiplication is simply taking linear combinations of the matrix columns!

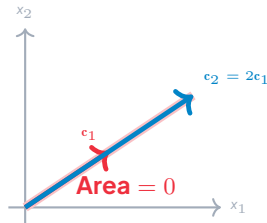
Hero Visualization: Determinants & Information Collapse

Non-Singular Matrix ($\det(A) \neq 0$)



Properties: Area is scaled by $|\det(A)|$. Full Rank (Rank = 2). Transformation is **reversible** (A^{-1} exists).

Singular Matrix ($\det(A) = 0$)



Properties: Area collapsed to zero. Rank deficient (Rank = 1). Null space is 1D line (Nullity = 1). **Irreversible** information loss.

Production Checkpoint: Solving Linear Systems in NumPy

The Anti-Pattern: Explicit Matrix Inversion

```
1 # DO NOT DO THIS IN PRODUCTION:
2 A_inv = np.linalg.inv(A) # 0(n^3) ops
3 x = A_inv @ b           # Severe roundoff errors!
```

Flaw: Explicit inversion is $3 \times$ slower and numerically unstable near ill-conditioned matrices ($\kappa(A) \gg 1$).

The Enterprise Pattern: LU / LAPACK Solve

```
1 # PRODUCTION-GRADE LAPACK SOLVER:
2 # Solves Ax = b via LU decomposition
3 x = np.linalg.solve(A, b)
4
5 # Verify condition number before solving
6 cond = np.linalg.cond(A)
7 if cond > 1e12:
8     raise ValueError(f"Ill-conditioned A: {cond}")
```

Important Always use `np.linalg.solve`
over `np.linalg.inv`.

TOPIC 03

Finding Axes of Maximum Truth: Eigen & SVD

Invariant directions under transformation, spectral theorem, and PCA from scratch.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Identify invariant directions** geometrically where linear transformations act as pure scalars ($\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$).
- **Diagonalize symmetric operators** using the Spectral Theorem on data covariance matrices $\Sigma = \frac{1}{n}\mathbf{X}^T\mathbf{X}$.
- **Deconstruct SVD geometrically** as a 3-step pipeline: Rotation (V^T) $\rightarrow$ Scaling (Σ) $\rightarrow$ Rotation (U).
- **Implement PCA from scratch** using Singular Value Decomposition to compress high-dimensional feature spaces.

The High-Dimensional Redundancy Dilemma

The 50-Feature Multicollinearity Curse

- Customer transaction dataset contains $d = 50$ numeric columns.
- Many features (e.g., monthly spend, annual income, credit limit) are heavily correlated.
- **Curse of Dimensionality:** Models overfit, distance metrics degrade, latency escalates.
- **Challenge:** How do we extract the top 3 orthogonal directions in space that preserve 95% of all data variance?

The Discovery: Invariant Axes of Variance

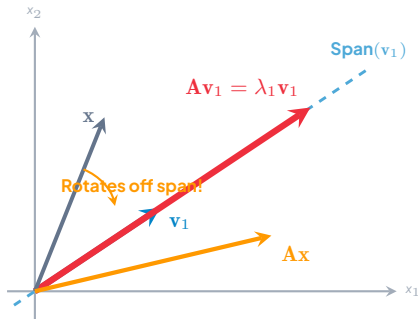
- Under transformation, most vectors rotate out of their original line of action.
- **Eigenvectors** never rotate; they only stretch by factor λ :

$$A\mathbf{v} = \lambda\mathbf{v}$$

- On the data covariance matrix, the top eigenvector points along the line of **maximum spread / variance!**

Important Eigenvectors find the natural coordinate system of the data.

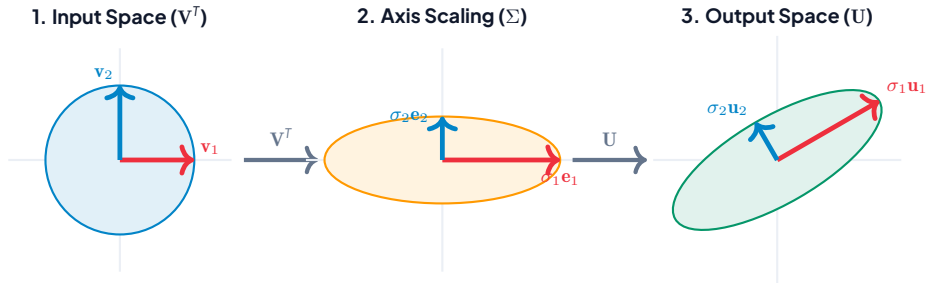
Hero Visualization: Eigenvectors as Invariant Directions



Invariant Directions Under A

- **General Vector x :** Multiplied by A , changes both length **and** direction ($Ax \nparallel x$).
- **Eigenvector v :** Multiplied by A , remains on its original span line ($Av \parallel v$).
- **Eigenvalue λ :** The scalar stretch factor along the invariant line.
- **Spectral Theorem:** Any symmetric matrix $S = S^T$ has an orthonormal basis of eigenvectors ($S = Q\Lambda Q^T$).

Hero Visualization: SVD Geometric Decomposition



Important

Every real matrix $X = U\Sigma V^T$: Orthogonal Rotation $\rightarrow$ Axis Stretch $\rightarrow$ Orthogonal Rotation.

Production Checkpoint: Principal Component Analysis via SVD

PCA Mathematical Mechanics

For zero-centered data $\mathbf{X} \in \mathbb{R}^{N \times d}$:

1. Compute SVD: $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
2. Right singular vectors $\mathbf{V} \in \mathbb{R}^{d \times d}$ are the principal loading directions.
3. Project to k dimensions:
 $\mathbf{Z} = \mathbf{X}\mathbf{V}_{:,k} = \mathbf{U}_{:,k}\mathbf{\Sigma}_{:,k,k}$.
4. Explained variance ratio: $\frac{\sigma_i^2}{\sum \sigma_j^2}$.

NumPy Implementation from Scratch

```
1 import numpy as np
2
3 def pca_from_scratch(X: np.ndarray, k: int):
4     # 1. Center the features (zero mean)
5     X_centered = X - np.mean(X, axis=0)
6
7     # 2. Economy SVD decomposition
8     U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
9
10    # 3. Project data onto top-k components
11    Z = X_centered @ Vt[:k].T # shape: (N, k)
12
13    # 4. Compute explained variance ratio
14    explained_var = (S**2) / np.sum(S**2)
15    return Z, Vt[:k], explained_var[:k]
```

Session Summary

01 How Machines “See” Data: Vectors & Norms

High-dimensional feature space, dot products as projections, and L1 vs L2 geometry.

02 What is a Matrix, Really? Linear Transformations

Matrices as spatial deformation operators, basis shifts, determinants, and rank.

03 Finding Axes of Maximum Truth: Eigen & SVD

Invariant directions under transformation, spectral theorem, and PCA from scratch.

Professional Program in Enterprise Data Science & Agentic AI with AWS — Modular Session Architecture